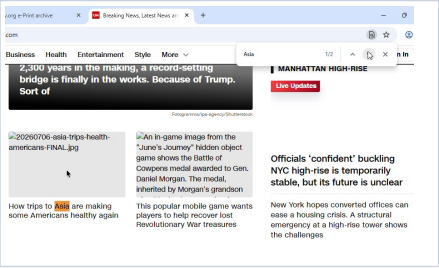
TEAM 21 · HACKATHON FINAL SUBMISSION · 17 OCTOBER 2026

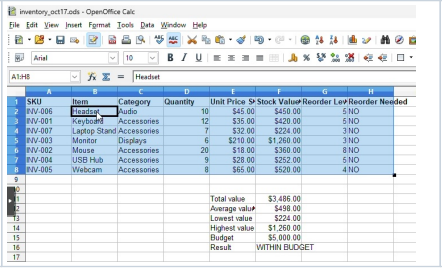
Human-Like Windows Desktop Agent

Implementation report: our approach, the architecture, and the function of each agent

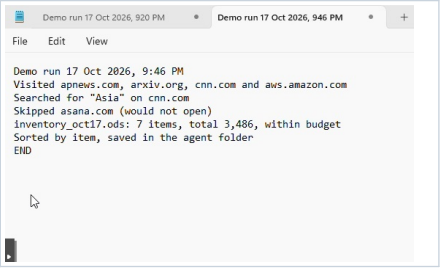
Windows 11 VM · Google Chrome · OpenOffice Calc 4.1.16 · Notepad · Python 3.12



Chrome: find-in-page on CNN



Calc: the finished, sorted workbook



Notepad: the notes, ending with END

The final demonstration at a glance

Started	From the **start agent** desktop shortcut (pythonw.exe), Full demo chosen in the launcher
Duration	**14 min 15 s** from Start to the last line of the notes; the limit is 15 minutes
Browsing	AP News, arXiv, CNN and AWS read; asana.com skipped (certificate warning with no Proceed link); one find-in-page search ("Asia" on CNN); one return to an earlier tab
Spreadsheet	**inventory_oct17.ods** in Desktop\agent: 7 records, formulas, two corrections, a totals block (SGD 3,486, within the SGD 5,000 budget), bold header, currency format, sorted by Item
Notes	Six lines describing the run, then **END**; left open and unsaved for the next person
Trace	Every application launched by Explorer from its desktop icon; **no files written by the agent** (only Calc wrote the workbook)

Submitted files: main.py, browser_action.py, inventory_action.py, notes_action.py, gui_support.py, save_dialog.py, domain.py, config.json, inspect_ui.py, requirements.txt, README.md and the demo recording *demo run .mov*.

Contents

1 Summary . . . 3

2 The task, the environment and the criteria . . . 3

3 Architecture . . . 4

Life of a run . . . 5

4 The agents and their functions . . . 5

4.1 Orchestrator (main.py) . . . 5

4.2 Browser agent (browser_action.py) . . . 5

4.3 Spreadsheet agent (inventory_action.py) . . . 6

4.4 Notes agent (notes_action.py) . . . 7

4.5 Human-input layer (gui_support.py) . . . 7

4.6 Domain logic (domain.py) and the Save dialog helper (save_dialog.py) . . . 8

5 Approach: behaving like a person . . . 8

6 Approach: process authenticity . . . 9

7 Approach: resource efficiency . . . 9

8 Robustness, timing and safety . . . 10

9 How we developed and tested it . . . 10

10 The final demonstration run . . . 11

11 Limitations and future work . . . 13

Appendix A Running the agent . . . 14

Appendix B Main settings (config.json) . . . 14

1 Summary

We built a Python agent that carries out an ordinary office session of up to fifteen minutes on the organisers' Windows 11 virtual machine, the way a person at the keyboard would: through the visible user interface, with a real mouse pointer and real key presses. It browses news and technology homepages in Google Chrome, builds a small stock-inventory workbook with formulas, totals and formatting in OpenOffice Calc and saves it, and finally types a short account of the session into Notepad, ending with the line END and leaving it on screen for whoever comes back to the computer.

The design follows the four judging criteria. Every application is started the way a user starts it, by double-clicking its desktop icon, so Windows Explorer is the parent of every process the agent uses, and the agent writes no files of its own. Its timing comes from distributions measured on a recording of a person browsing by hand and on our own test runs in the VM, with a new "personality" drawn for every run. The work involves three applications and many-step interactions: certificate warnings, pop-ups, tabs, find-in-page, dialogs, formulas and sorting. And the agent is light: it waits for events rather than polling, and reads the accessibility tree only in small, bounded walks.

The agent is organised as three task agents (browser, spreadsheet and notes) run in turn by an orchestrator, on top of a shared human-input layer ("the hands") and a library of pure domain logic ("the knowledge"). It was developed over seven rounds of testing in the VM, each screen-recorded and analysed frame by frame, and the project repository holds 424 automated tests that run without Windows, including a simulated OpenOffice Calc. Section 10 walks through the final demonstration run.

2 The task, the environment and the criteria

Environment. The agent runs inside a Proxmox virtual machine (Windows 11 at 1280 × 800, reached through a noVNC console) with Google Chrome, Apache OpenOffice Calc 4.1.16 and Notepad. Only site homepages load. Many HTTPS sites first show Chrome's "Your connection is not private" page; the organisers' rule is to click *Advanced*, then *Proceed*, and to skip a site that offers no Proceed link. Judges examine a run with Sysmon, Process Explorer and Event Viewer, not only on screen.

The criteria and our response. Table 1 summarises how each criterion shaped the implementation; Sections 5 to 8 give the detail.

Criterion	Weight	What we did
Process authenticity	30%	The agent starts from a desktop shortcut (pythonw.exe, no console). Chrome, Calc and Notepad are launched by double-clicking their desktop icons, so Explorer starts them. Everything that changes a page or a document is a real mouse or keyboard event. The agent writes no log, report or cache file during a run.
Human entropy and timing	30%	Pointer movement follows Fitts' law with a minimum-jerk speed profile, a shallow arc, tremor and occasional overshoot. Typing has per-key gaps by key type, held keys, Shift timing and corrected slips. Pauses are heavy-tailed (lognormal). Pace, wheel speed, typo rate and pause length are drawn anew for every run. No fixed sleeps.
Complexity of action	20%	Chrome: typed addresses, certificate warnings, consent and offer pop-ups, reading with the wheel and keys, find-in-page, tabs. Calc: an eight-column table with formulas, edits, copy and paste, a totals block, bold and currency formatting, sorting through the Data menu, Save As through the Windows dialog. Notepad: a written summary of the session.
Resource efficiency	20%	Waits are event-driven; accessibility-tree walks are bounded and skip page content when only the browser frame is needed; the screen is read in small rectangles; no background threads. The agent process averaged about 0.1% CPU and 60 MB in VM measurements.

Table 1. The four judging criteria and how the implementation addresses them.

3 Architecture

The program is a set of small modules with one job each (Figure 1). The orchestrator owns the run and its time budget; each task agent owns one application; all of them act only through the human-input layer, which is the single place where mouse and keyboard events are produced; and the domain module holds everything that can be computed without Windows, which is why most of the logic can be tested on any computer.

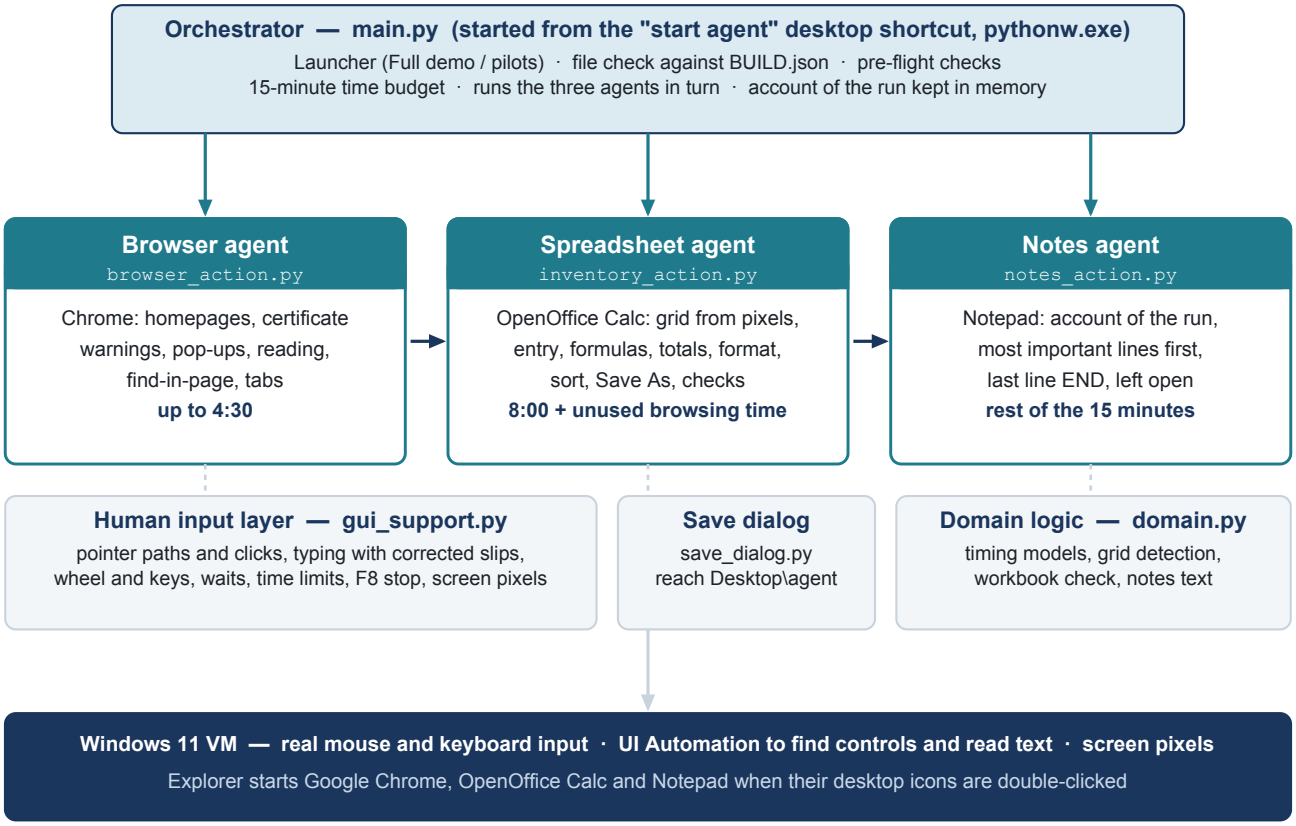


Figure 1. Architecture. The three task agents run in turn; they share the human-input layer, the Save dialog helper and the domain logic.

File	Lines	Role
`main.py`	358	Orchestrator: launcher window, file check, pre-flight checks, time budget, running the phases, result
`browser_action.py`	1,599	Browser agent (Chrome)
`inventory_action.py`	1,219	Spreadsheet agent (OpenOffice Calc)
`notes_action.py`	170	Notes agent (Notepad)
`gui_support.py`	707	Human-input layer: pointer, clicks, typing, scrolling, waits, limits, screen reading
`save_dialog.py`	183	The Windows Save dialog, driven by clicks (shared by Calc)
`domain.py`	544	Pure logic: timing models, grid detection, workbook check, notes text
`config.json`	—	All tunable settings and the 27 homepages
`inspect_ui.py`	92	Diagnostic tool, run by hand only (dumps controls; checks the grid reader)

Table 2. Source files (4,872 lines of Python). Dependencies: pyautogui, pywinauto and pywin32.

Life of a run

- A person double-clicks the **start agent** desktop shortcut, which runs `main.py` with pythonw.exe, so no console window appears. The launcher (Figure 2) offers the **Full demo** and two pilots (browser only, or spreadsheet and notes only).
- Before any input is sent, the orchestrator checks its own files (every module the run needs is loaded, and the files are compared with the `BUILD.json` list of SHA-256 hashes shipped with the build) and the desktop (Chrome, Calc and Notepad must not be open yet; the shortcuts must exist; Caps Lock must be off; the keyboard layout should be English (US)).
- The three phases follow with hard deadlines (Table 3). Finishing the spreadsheet has priority, so browsing time that is not used is given to Calc, and the notes take whatever is left before the 15-minute mark.
- A complete run ends quietly with the notes on screen. If something went wrong, a box gives the status, the runtime and the error.

Phase	Time allowed	Rule
Browsing	up to 4:30	`browser_phase_seconds = 270`; a new site is not started with under 30 s left
Spreadsheet	8:00 + unused browsing time	`full_phase_seconds = 480`, but never later than 15:00 minus the notes' reserve of 1:30
Notes	the rest, at least 1:30	Ends 5 s before the 15-minute mark; skipped if under 45 s remain

Table 3. The 15-minute time budget of a Full demo.

4 The agents and their functions

4.1 Orchestrator (main.py)

- **Launcher.** A small Tk window to choose the mode; the budget for the spreadsheet comes from the configuration, so nothing has to be typed before a run.
- **File check.** Loads each module the chosen mode needs and compares every module with the build manifest. A module that cannot be loaded stops the run before Chrome opens, with a message saying which file to copy again (a copy into the VM once left one file empty and broke a run four and a half minutes in).
- **Pre-flight.** Applications already open, missing shortcuts, Caps Lock and the keyboard layout are reported with Retry, Start anyway or Cancel.
- **Phases and failures.** Runs browsing, the spreadsheet and the notes; a failure in browsing does not stop the spreadsheet. The final status is complete, partial (with the reason), stopped or failed.
- **Memory of the run.** What each phase did (sites, searches, pop-ups, the workbook and its totals) is kept in memory and handed to the notes agent; nothing is written to disk.

4.2 Browser agent (browser_action.py)

The browser agent reads homepages the way a curious person does, a few minutes at a time, and copes with what the VM puts in the way.

- **Which sites.** 27 homepages (news, technology, finance, research) are shuffled for every run. Sites known from earlier VM runs to be of no use (no Proceed link, the site's own 403 or 404 page, a blank page) are kept to the end of the list. Each address is typed at most once per run.
- **Opening a site.** A new tab is opened with Chrome's **+** button (Ctrl+T as fallback); the pointer goes to the address bar and the address is typed, with a slip now and then that is noticed and corrected. A tab whose site would not load is reused for the next address.

- **Certificate warnings.** The agent first makes sure the warning names the host it asked for (a warning left over from the previous site is not acted on). It reads the warning for about a second, clicks *Advanced* (once more if the details stay closed) and clicks *Proceed to <host> (unsafe)* only when the host matches the approved homepage. Pages with no Proceed link (HSTS) and the site's own error pages are skipped at once; a passing problem gets one retry.
- **Pop-ups.** Consent and cookie notices are accepted (*Agree*, *Accept all*; never Reject, Manage, Settings or Sign in). Other pop-ups, such as Bitly's "Want to try Bitly for free?" offer, are closed with their X, with Esc as the fallback, and never with their sign-up button. Only controls inside the page are considered (Chrome's own tab and find-bar Close buttons never are), and a click that leaves the homepage is undone with Alt+Left.
- **Reading.** Each page gets an engagement style: skim (2–4 sections, 22% of pages), normal (5–8, 58%) or deep (9–13, 20%), within 30–45 s per site. The agent scrolls with flicks of the wheel, sometimes with Page Down or Space, rests the pointer on a headline or picture, holds one item of interest for 4–8 s, now and then zooms in and back, reloads, or double-clicks a word, and often flicks back to the top at the end.
- **Find in page.** On at most two sites per run, never two in a row, the agent looks a word up with Chrome's find bar (Ctrl+F): a word that really appears below the page's menu, so the page visibly moves to it. It clicks the find bar's *Next* arrow with the mouse, rests the pointer on the highlighted word, then closes the bar with Esc (Figure 4).
- **Tabs.** Up to three tabs. The agent goes back to the first site's tab at most twice a run (never twice in a row), sometimes with Ctrl+Tab instead of a click.
- **Safety.** Only the approved homepages are visited: links to products, sign-in or registration are never activated.

4.3 Spreadsheet agent (inventory_action.py)

OpenOffice Calc 4.1.16 exposes neither its cells, nor its Name Box, nor its menus to UI Automation. The spreadsheet agent therefore works the way eyes do: it reads the grid from the screen. A few screen rectangles are copied in one operation, and the column and row headers of the selected cell, drawn in the highlight colour, together with the one-pixel separator lines, give the position and size of every cell. The pointer then clicks cells and drags ranges, and the agent checks from the headers that the selection is the one it wanted (the keyboard is the fallback).

The work is ordered essentials first, so that a slow VM loses only the polish:

#	Step	How it is done
1	Launch and headers	Calc from its desktop icon; eight headers typed with Tab
2	Save As	Through the Windows dialog: Desktop, then the agent folder, by clicks; only the name `inventory_<mon><day>.ods` is typed (Figure 5)
3	Records	The first record typed in full; SKUs filled down; repeated parts (categories, formulas) copied and pasted, as people do
4	Corrections and a new record	Two values edited (USB Hub quantity 4 → 9, Monitor price 220 → 210); INV-007 Laptop Stand added below
5	Check	Values and formulas compared with what was intended; missing formulas repaired
6	Totals	Total, average, lowest, highest, budget and an IF budget test; then File › Save, visibly
7	Polish (if it fits)	Header row bold; currency format; sort by Item through Data › Sort
8	Review until the cap	Walk the cells, check the totals, try a quantity and put it back, look an item up
9	Wrap-up	File › Save, close Calc; the saved file is opened as a ZIP and checked

Table 4. The order of work in Calc.

- **Keeping to time.** At five checkpoints the agent compares its progress with the times measured for each part of the work in a VM pilot and projects when the workbook will be done. If it would not finish in time, it hurries: shorter pauses, a faster pace (not below 0.75), and no optional steps until it is back on time. Polish only runs if it fits before the cap.
- **Reaching the right folder.** The Save dialog's address bar is read first (Windows reopens the folder used last, and nothing is clicked if it already shows the agent folder). The left pane is scrolled when Desktop is out of view. Save is never pressed while the dialog shows another folder: the full path is typed instead. A "replace?" question outside the agent folder is answered No, and only a file written by this save is accepted as the workbook.
- **Checking the result.** The saved .ods file is read (a ZIP holding XML) and must contain the seven records, the right values and formulas, the total of 3,486 and the correct budget result; sorting is reported. An unfinished table is reported as partial, never as complete.

SKU	Item	Category	Qty	Unit price	Stock value	Reorder level	Reorder needed
INV-006	Headset	Audio	10	45.00	450.00	5	NO
INV-001	Keyboard	Accessories	12	35.00	420.00	5	NO
INV-007	Laptop Stand	Accessories	7	32.00	224.00	3	NO
INV-003	Monitor	Displays	6	210.00	1,260.00	3	NO
INV-002	Mouse	Accessories	20	18.00	360.00	8	NO
INV-004	USB Hub	Accessories	9	28.00	252.00	5	NO
INV-005	Webcam	Accessories	8	65.00	520.00	4	NO

Table 5. The finished table, sorted by Item (prices in SGD). Stock value is `=D*E` *and Reorder needed is* `=IF(D<G;"YES";"NO")`*. Below the table: total value* `=SUM(F2:F8)` *= 3,486.00, average 498.00, lowest 224.00, highest 1,260.00, budget 5,000.00, and* `=IF(F11<=F15;"WITHIN BUDGET";"OVER BUDGET")`*.*

4.4 Notes agent (notes_action.py)

- **Opening Notepad.** By double-clicking its desktop icon (Win+R and "notepad" if the shortcut is missing). Windows 11 Notepad reopens unsaved notes of earlier sessions, so a new tab is started when the page is not empty.
- **What it writes.** A short account built from the run's memory: the date and time, the sites visited, pop-ups accepted or closed, the word searched for, the sites skipped, the workbook with its totals, and where it was saved (Figure 6).
- **Fitting the time.** Each line has a priority (the run, the sites and the spreadsheet come first). Before each line the choice is made again with the typing speed measured so far, so when typing runs slower than planned a less important line is left out, rather than an important one being cut in the middle.
- **END.** The last line is always **END**, with time kept back for it, to mark the end of the script. The notes are left open and unsaved, so whoever returns to the computer can read what was done.

4.5 Human-input layer (gui_support.py)

The "hands" of all three agents. It turns intentions ("click this control", "type this value") into human-shaped input events and protects the run:

- **Pointer.** `move` follows a timed path from the domain module (Section 5); `click` hovers before pressing, checks that the target is still there and that the pointer is still on it (a host mouse resting over the VM console once nudged the pointer), holds the button for 50–140 ms, and pauses to look at the result. `drag` and `wheel` work the same way.

- **Keyboard.** `type_text` sends real key-down and key-up events with human gaps, holds and Shift timing, and makes and corrects slips; it refuses to type if another window has taken the focus. Arrow and navigation keys are sent with the extended-key flag, and modifier keys left held by anything else are released first.
- **Applications.** `launch_shortcut` shows the desktop (Win+D), finds the icon by name and double-clicks it, then waits for the window and maximises it.
- **Control.** Event-driven waits; a hard time limit per phase (any input after the deadline raises a `TimeUp` that ends the phase cleanly); F8 or the pointer in a screen corner stops the run at once.
- **Seeing.** Accessibility (UI Automation) is used to find controls and read text and addresses; `grab` reads a screen rectangle with one GDI copy for the Calc grid.

4.6 Domain logic (domain.py) and the Save dialog helper (save_dialog.py)

`domain.py` holds pure functions with no Windows dependency: the timing models (pauses, Fitts' law, pointer paths, key timing, neighbouring keys for slips), the grid detection from pixels, reading and validating the .ods file, choosing and wording the notes, and the URL safety rules. `save_dialog.py` drives the Windows Save dialog: it reads the address bar, finds the left-pane item that is really in view, rolls the pane with the wheel when needed, double-clicks the folder, answers the replace question and picks a free file name.

5 Approach: behaving like a person

The human baseline is a 223-second recording of a person browsing the same kind of sites by hand, analysed second by second. It gave the pointer travel times (short adjustments 0.20–0.42 s, medium moves 0.35–0.72 s, long ones 0.60–1.08 s), how long a view is read (a new section every 1–3 s, 3–5 s for distinct content, about 9 s for an item of interest), and habits such as clicking Chrome's + button and tabs, and pausing over the text revealed by *Advanced*. Measurements from our own VM recordings (typing speed, Save As, row entry) completed the picture. Table 6 lists the models.

Behaviour	Model	Values
Pointer travel time	Fitts' law with lognormal noise	$(0.17 + 0.15 \cdot \log_2(\text{distance} / \text{width} + 1)) \cdot \text{LogN}(0, 0.25) \cdot \text{pace}$, kept within 0.18–1.7 s
Pointer path	Minimum-jerk profile with an early speed peak; shallow arc; tremor	Samples at irregular 70–140 Hz; 40% of moves over 90 px overshoot or undershoot by up to 6% and correct
Clicking	Hover, then press	Hover lognormal, median 0.55 s; button held 50–140 ms; double-click gap 70–160 ms
Key gaps	By key type, times a pace per word	Letters and digits 0.12–0.30 s, after a word 0.25–0.65 s, symbols 0.35–0.95 s; × 0.9–1.25 per word
Keys held, Shift	Real key down and up	Held 50–120 ms; Shift adds 0.15–0.40 s
Hesitation	Random extra pause	3.5% of keys, 0.4–1.1 s
Slips	A neighbouring key on a QWERTY keyboard, noticed and fixed	Noticed after 0.45–0.9 s, Backspace, retyped. 30% of addresses; in Calc 4–30% of cells by column; 20% of note lines; never in formulas or paths
Pauses	Lognormal (right-skewed, about twice the spread of a uniform range)	Thinking: median 0.67 s, up to 2.5 s. Reading: median 6 s, 3–12 s
Scrolling	Wheel flicks of whole notches	2–4 flicks per section; 15% of sections with Page Down or Space
Personality per run	Drawn when a run starts	Pace 0.85–1.2, wheel speed 0.8–1.3, typo rate 0.8–1.4, pauses 0.85–1.25

Table 6. Timing and movement models (domain.py and gui_support.py). "Pace" is the run's personal speed.

- **No fixed delays.** pyautogui's built-in pause is switched off (`PAUSE = 0`); every wait is either drawn from a distribution or waits for something to happen (a window, a dialog, a file).
- **No two runs alike.** The order of sites, the reading style of each page, which page gets a search, the slips and the run's personality all change from run to run.
- **Habits, not only noise.** The agent pauses to read a warning before clicking, looks at a pop-up before closing it, copies and pastes repeated values in the spreadsheet, saves after milestones, reviews its totals, and tries a what-if value before putting it back.

6 Approach: process authenticity

- **Started like a user program.** The agent is launched from a desktop shortcut that runs pythonw.exe, so Explorer is its parent and no console window appears. The pre-flight check refuses to start while Chrome, Calc or Notepad is already open, so each is started fresh by the agent.
- **Applications started by Explorer.** The agent never starts a process itself (no subprocess calls). It shows the desktop and double-clicks the application's icon, so Explorer starts Chrome, Calc and Notepad and is their parent; the Win+R fallback for Notepad also belongs to Explorer.
- **Real input only.** Pages and documents are changed only by mouse and keyboard events sent to visible windows: no scripts in web pages, no editing of files behind the applications' backs. The workbook is created by Calc and saved through Calc's own Save As dialog.
- **No files of its own.** The agent writes nothing to disk during a run: no logs, no report, no history and no Python cache (`sys.dont_write_bytecode` is set before any module is loaded). Sysmon's file-create events (ID 11) therefore show pythonw.exe creating nothing; the only document created is the workbook, written by Calc, as when a person saves it. Chrome and Calc keep their usual profile files.
- **Honest about failure.** A run that did not finish says so (partial, stopped or failed); the agent never claims a complete workbook it could not verify.

7 Approach: resource efficiency

- **Waiting, not spinning.** Most of a run is deliberate pauses, during which the agent sleeps. Waits for windows and dialogs poll every 0.4 s with timeouts.
- **Small accessibility reads.** Walks of the accessibility tree are bounded in depth, number of nodes and time; when only the browser frame is needed (tabs, address bar, find bar), the page itself is skipped. The page document is cached for a few seconds, one walk serves both the consent check and the pop-up check, and control names are read only for controls on screen (a page such as CNN has hundreds of links).
- **Small screen reads.** The Calc grid is found by copying a few screen rectangles in one GDI operation each, not by taking full screenshots in a loop.
- **Nothing in the background.** No helper threads and no file output: an early version sampled its own CPU into a file every 5 s; it was removed.
- **Measured.** In a VM run the agent process averaged 0.11% CPU (peak 0.86%) with a working set of about 60 MB. During the final demonstration the Proxmox summary showed the whole VM at about 10% of its 4 vCPUs while browsing and 3.7% while the notes were typed, with 2.1–3.3 GiB of its 8 GiB in use.

8 Robustness, timing and safety

Risk	What the agent does
Running past 15 minutes	Hard deadlines per phase; input after a deadline raises TimeUp and the phase wraps up; the spreadsheet hurries when behind; the notes drop minor lines and always end with END
A slower VM	Checkpoints compare progress with pilot timings; optional steps are skipped first; essentials are always done first and saved
Wrong or missing files in the VM	All modules loaded and compared with the build manifest before any input
Sites that do not load	Warning matched to the requested host; Advanced clicked again if needed; HSTS and the site's own error pages skipped; one retry for passing failures; dead tabs reused
Pop-ups	Consent accepted, offers closed with their X or Esc; checked when a page opens, after a stuck scroll, after a search and on a return to a tab
OpenOffice quirks	Ctrl+D is not Fill Down in OpenOffice (it opens a list), so formulas are copied and pasted; pasted cells stay marked, so they are left with a click or an arrow key
Save dialog in the wrong folder	Address bar read; visible Desktop clicked; never saved elsewhere (full path typed); no replacing outside the agent folder; only a freshly written file accepted
Interference	Pointer checked before each press; focus checked before typing; stuck modifier keys released; F8 or a screen corner stops everything at once

Table 7. Main risks and how the agent handles them.

9 How we developed and tested it

We could not run the VM from our laptops, so every test run in the VM was screen-recorded and analysed frame by frame; each round produced a list of observed problems, a fix, and new tests that reproduce the problem. Table 9 lists the main lessons.

Round	What the recording showed	What we changed
1	Range selection never worked, so everything was retyped; no time limit; the whole path typed in Save As	Mouse-led Calc with the grid read from pixels; extended-key arrows; time caps; Save As by clicks
2	Ctrl+D opened OpenOffice's Selection List; Tab after a paste overwrote the pasted cell	Formulas copied and pasted; pasted cells left with a click or an arrow
3	Inventory pilot complete in 7.7 min; the host's mouse nudged the VM pointer; a Full demo left the totals unfinished; CNN's consent pop-up; a tab revisited three times	Pointer check before each press; spreadsheet first with hurry mode; consent accepted; at most two returns to a tab
4	Workbook saved in the wrong folder (left pane scrolled); a search on every site; a temporary row costing time	Save dialog pane scrolled until Desktop is in view; find bar on at most two sites; temporary row removed; notes left open
5	A file copied wrongly into the VM crashed a run at 4.5 min; Bitly's offer pop-up stayed open	File check with a build manifest before any input; offers closed with their X
5b	A team member's Sysmon review: pythonw.exe wrote logs every few seconds	All log, report, history and cache files removed
6	Notes cut mid-line (typing slower than planned); the workbook saved in This PC	Typing model from measured speed, lines re-chosen before each one, END line; never save outside the agent folder
Final	Complete run in 14 min 15 s	—

Table 9. Lessons from the VM test rounds.

- **Automated tests.** 424 tests (Python unittest) run on any computer: contract tests drive each agent through fake UI Automation controls (a fake Chrome with certificate pages, pop-ups and a find bar); a simulated OpenOffice Calc (`sim_calc.py`) reproduces the grid, the Save As dialog and OpenOffice's quirks with a simulated clock and a slowness factor, to check that the workbook completes within eight minutes even on a VM 1.3 times slower; and checks of the time budget, the notes, the file check and the absence of file writes.
- **Builds.** A build script packs the files at the top of a ZIP with the `BUILD.json` manifest, so the copy in the VM can be checked before each run.

10 The final demonstration run

The submitted recording (*demo run .mov*, 14 min 32 s) shows a Full demo on the VM on 17 October 2026, started at 9:46 PM. Times below are times in the recording.

Time	What happens
0:04	**start agent** is double-clicked on the desktop (its tooltip shows pythonw.exe)
0:06	The launcher: Full demo, Start (Figure 2)
0:14	Chrome opened from its desktop icon
0:17–0:33	**asana.com**: certificate warning; Advanced reveals no Proceed link, so the site is skipped (Figure 3)
0:33–1:30	**apnews.com** typed into the same tab and read
1:32–2:08	**arxiv.org** in a new tab
2:11–3:20	**cnn.com**; at 2:55 the word "Asia" is looked up with the find bar and the page moves to it (Figure 4)
3:25–3:45	A return to the AP News tab
3:47–4:40	**aws.amazon.com** in a new tab
4:47	Chrome closed; Calc opened from its desktop icon
5:02–5:52	Headers typed; Save As: the dialog already shows Desktop\agent, so only the name is typed (Figure 5)
6:00–9:05	Records INV-001 to INV-007 with copy and paste, formulas and the two corrections
9:15–11:13	Totals block; File › Save
11:18–11:33	Currency format and bold header; sort by Item through Data › Sort (Figure 7)
11:38–12:18	Review (a quantity tried and put back), File › Save, Calc closed
12:23–14:21	Notepad opened from its icon; six lines typed, then END (Figure 6)

Table 8. Timeline of the final demonstration.

Outcome. The run took 14 minutes 15 seconds from Start to END, inside the 15-minute limit, and every task completed: four homepages read, one site correctly skipped, one search, one return to a tab; the workbook complete, formatted, sorted and saved in the agent folder; the notes complete and ending with END. No result box appeared, as intended for a complete run.

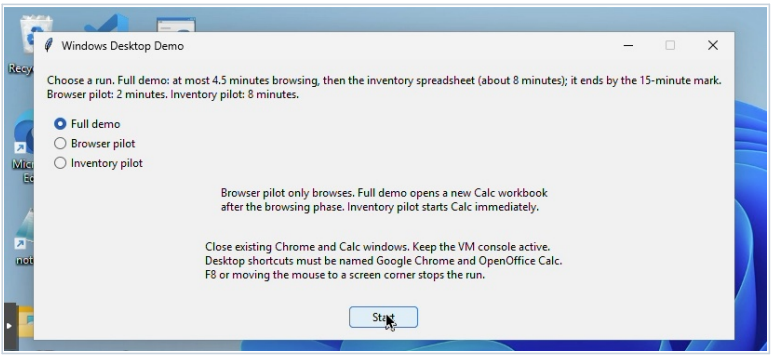


Figure 2. The launcher, opened from the start agent shortcut (0:06).

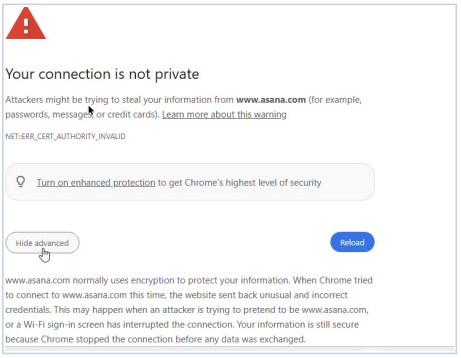


Figure 3. asana.com: Advanced shows no Proceed link, so the site is skipped (0:30).

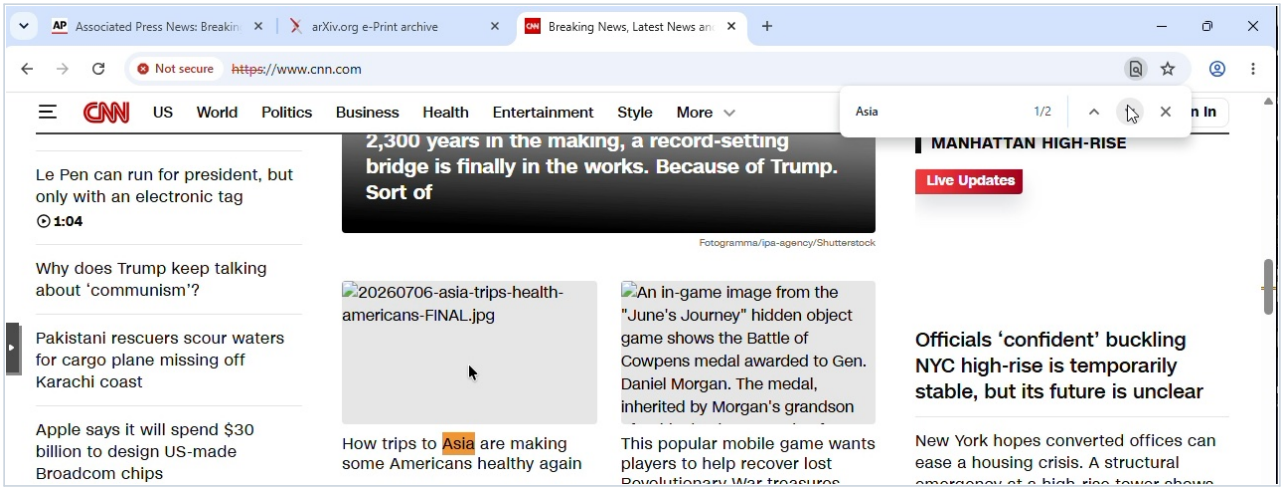


Figure 4. CNN: the find bar shows "Asia" and the page has moved to the highlighted match in an article (3:05).

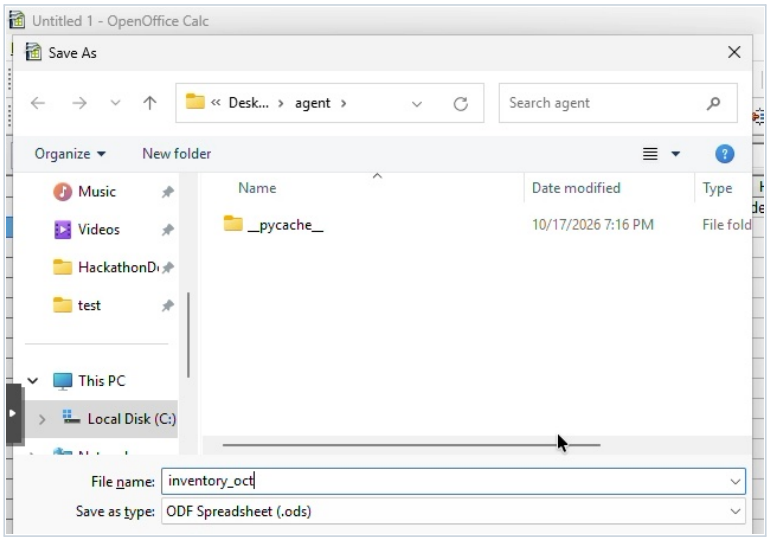


Figure 5. Save As (5:44). The dialog opened in Desktop › agent, the folder used last, so the agent clicked nothing and typed only the file name, inventory_oct17.ods. Had the address bar shown another folder, the agent would have clicked Desktop and the agent folder, or typed the full path, before pressing Save.

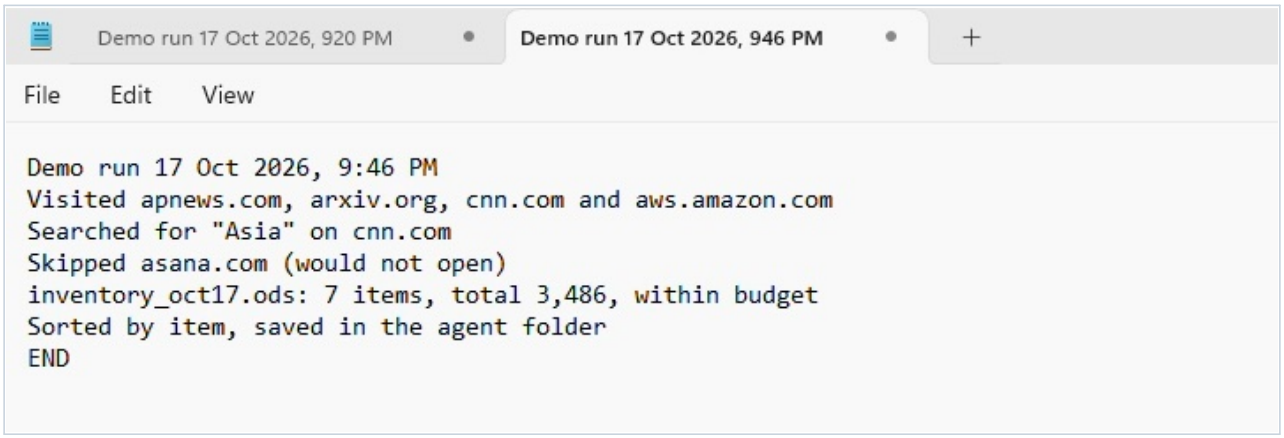


Figure 6. The notes at the end of the run: six lines describing what was done, then END (14:22). The earlier tab holds the notes of a previous run, reopened by Notepad.

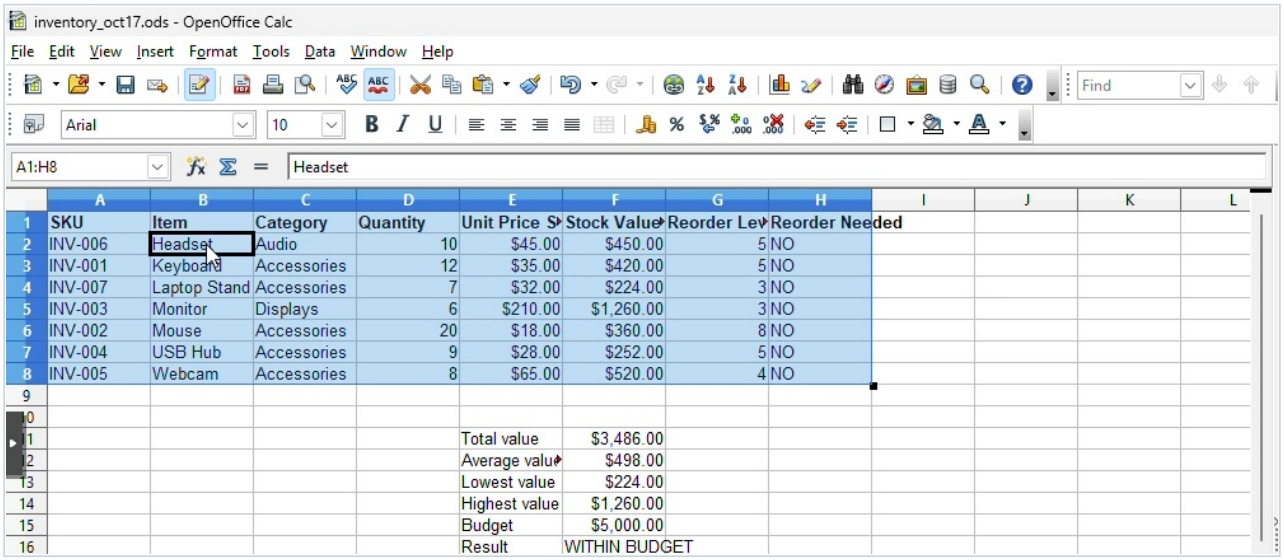


Figure 7. The finished workbook just after sorting by Item: currency format, bold header and the totals block, total SGD 3,486.00, within budget (11:30).

11 Limitations and future work

- **The VM's network.** Some homepages render only partly (a page that goes white once scrolled, a header with no body). The agent skips sites that cannot be passed, but it still spends a visit on a page that renders blank; detecting a blank render on screen and moving on sooner would leave time for one more site.
- **Seeing Calc by pixels.** The grid reader is tuned to the VM's screen (1280 × 800) and Calc's default colours; a different theme or zoom would need new measurements (the keyboard fallback still works).
- **Dialogs differ between Windows builds.** The Save dialog's accessibility tree is not the same everywhere; the agent therefore checks where the dialog is before saving instead of trusting its clicks.
- **No logs by design.** Without log files, problems are diagnosed from screen recordings and the result box. This is the price of a clean Sysmon trace.
- **Next steps.** Widen the spreadsheet columns at the end; point at formula ranges with the mouse; sometimes use toolbar buttons and the title-bar close button instead of shortcuts; check the process chain in Process Explorer on every run.

Appendix A Running the agent

- **Requirements.** Windows 11 with Python 3.12 and the packages in `requirements.txt` (pyautogui, pywinauto, pywin32); desktop shortcuts named **Google Chrome**, **OpenOffice Calc** and **Notepad**.
- **Install.** Copy all files into `C:\Users\User\Desktop\agent`. Create a desktop shortcut named **start agent** with the target `pythonw.exe "C:\Users\User\Desktop\agent\main.py"`.
- **Before a run.** Close Chrome, Calc and Notepad (the previous run's notes need not be saved), turn Caps Lock off, and keep the host's mouse away from the VM console.
- **Run.** Double-click **start agent**, choose **Full demo** (or a pilot) and press **Start**; then do not touch the keyboard or mouse. Hold **F8**, or push the pointer into a screen corner, to stop.
- **After the run.** The workbook is in the agent folder; the notes are on screen. A box appears only if something went wrong.

Appendix B Main settings (config.json)

Setting	Value	Meaning
browser_phase_seconds	270	Browsing time in a Full demo (4:30)
full_phase_seconds	480	Spreadsheet time (8:00), plus unused browsing time when carry_unused_browser_time is on
notes_phase_seconds	90	Time kept for the notes; they get everything left before total_demo_seconds
total_demo_seconds	900	The 15-minute limit of a Full demo
homepages	27 sites	Addresses with a keyword each; shuffled every run
browser_known_blocked	10 sites	Sites seen to be of no use in the VM, kept to the end of the list
browser_visit_seconds	30–45	Length of one visit
browser_find_probability	0.3	Chance of a find-in-page search on a site (at most browser_max_searches = 2)
browser_max_revisits	2	Returns to the first site's tab per run (chance 0.5 each time)
browser_typo_probability	0.3	Chance of a corrected slip when typing an address
browser_accept_consent / browser_close_popups	true	Accept consent notices; close other pop-ups with their X
keyboard_scroll_probability	0.15	Sections read with Page Down or Space instead of the wheel
calc_drag_select_probability	0.6	Ranges selected by dragging rather than with the keyboard
spreadsheet_typo_scale	1.25	Scale of the per-column slip chances in Calc
calc_wrap_up_seconds	20	Time kept at the end of Calc to save and close
default_budget_sgd	5000	Budget used in the totals block

Table 10. The main settings; every other value in config.json has a working default.